

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Repaso

Recursividad & Merge Sort

Merge Sort: Ordenamiento Eficiente

Ejercicios Guiados

Medición de Rendimiento

Aplicación: Catálogo Estelar

Consolidación y Retroalimentación

Conclusiones

Introducción y Repaso

Introducción y Repaso $\ni$ Recapitulación de la Sesión Anterior (P1)

- **Recursividad:** función que se llama a sí misma
 - Caso base + caso recursivo
 - Ejemplos: factorial, suma de listas
- **Bubble Sort:** complejidad $\mathcal{O}(n^2)$
 - Comparación de elementos adyacentes
 - Ineficiente para listas grandes
- **Búsqueda:** lineal $\mathcal{O}(n)$ vs binaria $\mathcal{O}(\log n)$

Conexión

Hoy aplicaremos recursividad para crear algoritmos de ordenamiento más eficientes.

- **Implementar** Merge Sort usando recursividad
- **Comparar** empíricamente algoritmos de ordenamiento
- **Medir** rendimiento con herramientas reales
- **Visualizar** escalamiento de complejidad
- **Aplicar** algoritmos a datasets astronómicos

Recursividad & Merge Sort

Recursividad & Merge Sort $\ni$ ¿Qué es recursividad?

- Una función que se llama a sí misma para resolver un problema más pequeño.
- Necesita: *caso base* + *llamada recursiva*.
- Ejemplo clásico: factorial, Fibonacci... ¡y algoritmos de ordenamiento!

Recursividad & Merge Sort $\ni$ Merge Sort en Python ($\mathcal{O}(n \log n)$)

```
1 def merge_sort(lst):
2     if len(lst) <= 1:
3         return lst                # caso base
4     mid = len(lst) // 2
5     left = merge_sort(lst[:mid])  # divide
6     right = merge_sort(lst[mid:])
7     return merge(left, right)     # conquista
8
9 def merge(left, right):
10    out, i, j = [], 0, 0           # apuntadores
11    while i < len(left) and j < len(right): # mezcla ordenada
12        if left[i] < right[j]:      # comparar
13            out.append(left[i])     # agregar menor
14            i += 1                  # avanzar en left
15        else:
16            out.append(right[j])    # agregar menor
17            j += 1                  # avanzar en right
18    return out + left[i:] + right[j:] # agregar restos
```

- Dos fases: división recursiva y mezcla ordenada.
- Estable (**preserva orden relativas de iguales**) y predecible.

Merge Sort: Ordenamiento Eficiente

Merge Sort: Ordenamiento Eficiente $\ni$ Merge Sort: ¿Por qué es mejor que Bubble Sort?

Problema con Bubble Sort

Con 10,000 elementos $\rightarrow$ 100,000,000 comparaciones

Tiempo: varios segundos o minutos

Ventaja de Merge Sort

Con 10,000 elementos $\rightarrow$ 133,000 comparaciones

Tiempo: milisegundos

Complejidad: $\mathcal{O}(n \log n)$ vs $\mathcal{O}(n^2)$

Diferencia práctica

Para 100,000 elementos: Bubble Sort tardaría **horas**, Merge Sort tardaría **segundos**.

Merge Sort: Ordenamiento Eficiente $\ni$ Merge Sort: Estrategia "Divide y Conquista"

Idea Principal

1. **Divide:** Partir la lista en dos mitades
2. **Conquista:** Ordenar recursivamente cada mitad
3. **Combina:** Mezclar las dos mitades ordenadas

Ejemplo visual con lista original:



División recursiva:

- Paso 1: 5 2 | 8 1 9
- Paso 2: 5 2 | 8 1 9
- Paso 3: 5 2 | 8 1 9

Merge Sort: Ordenamiento Eficiente $\ni$ Merge Sort: Visualización con Cajas

Lista original



División:



Mezcla ordenada:



Merge Sort: Ordenamiento Eficiente $\ni$ Merge Sort: Función de Mezcla (merge)

Paso 1: Entender cómo mezclar dos listas ordenadas

```
1 def merge(left, right):
2     """
3     Mezcla dos listas ordenadas en una sola lista ordenada.
4     Ejemplo: merge([2, 5], [1, 8])  $\rightarrow$  [1, 2, 5, 8]
5     """
6     resultado = []
7     i, j = 0, 0
8
9     # Comparar elementos de ambas listas
10    while i < len(left) and j < len(right):
11        if left[i] < right[j]:
12            resultado.append(left[i])
13            i += 1
14        else:
15            resultado.append(right[j])
16            j += 1
17
18    # Agregar elementos restantes
19    resultado.extend(left[i:])
20    resultado.extend(right[j:])
21    return resultado
```

Merge Sort: Ordenamiento Eficiente $\ni$ Merge Sort: Función Principal (Recursiva)

```
1 def merge_sort(lista):
2     """
3     Ordena una lista usando el algoritmo Merge Sort.
4     Complejidad:  $O(n \log n)$ 
5     """
6     # Caso base: lista de 0 o 1 elemento ya está ordenada
7     if len(lista) <= 1:
8         return lista
9
10    # Dividir la lista en dos mitades
11    mid = len(lista) // 2
12    left = lista[:mid]
13    right = lista[mid:]
14
15    # Ordenar recursivamente cada mitad
16    left_ordenada = merge_sort(left)
17    right_ordenada = merge_sort(right)
18
19    # Combinar las mitades ordenadas
20    return merge(left_ordenada, right_ordenada)
```

Elementos clave: Caso base, división, recursión, mezcla.

Ejercicios Guiados

Ejercicios Guiados $\ni$ Ejercicio 1:

Implementar Merge Sort



Enunciado

- Implementar las funciones `merge()` y `merge_sort()`
- Probar con la lista: `[64, 34, 25, 12, 22, 11, 90]`
- Imprimir cada nivel de recursión para visualizar el proceso
- Verificar que la lista final esté ordenada

Conceptos: Recursividad aplicada, caso base, mezcla de listas.

Aplicación: Base para ordenar cualquier tipo de datos.

Ejercicios Guiados $\ni$ Solución 1 de Referencia:

Implementación Básica



```
1 def merge(left, right):
2     resultado = []
3     i, j = 0, 0
4
5     while i < len(left) and j < len(right):
6         if left[i] < right[j]:
7             resultado.append(left[i])
8             i += 1
9         else:
10            resultado.append(right[j])
11            j += 1
12
13    resultado.extend(left[i:])
14    resultado.extend(right[j:])
15    return resultado
```

```
14 def merge_sort(lista):
15     if len(lista) <= 1:
16         return lista
17
18     mid = len(lista) // 2
19     left = merge_sort(lista[:mid])
20     right = merge_sort(lista[mid:])
21
22     return merge(left, right)
23
24 # Prueba
25 datos = [64, 34, 25, 12, 22, 11, 90]
26 ordenado = merge_sort(datos)
27 print(f"Original: {datos}")
28 print(f"Ordenado: {ordenado}")
```

Discusión: Observa cómo la recursión maneja automáticamente todos los niveles.

Ejercicios Guiados $\ni$ Ejercicio 2: Comparación de Algoritmos



Enunciado

- Crear función `comparar_algoritmos(n)` que:
 - Genere lista aleatoria de tamaño `n`
 - Mida tiempo de Bubble Sort y Merge Sort
 - Retorne ambos tiempos en segundos
- Probar con `n = 100, 500, 1000, 2000`
- Crear tabla comparativa de resultados

Conceptos: Medición de rendimiento, módulo `time`.

Aplicación: Entender complejidad algorítmica prácticamente.

Medición de Rendimiento

¿Cómo medir cuánto tarda un algoritmo?

```
1 import time
2
3 # Iniciar cronómetro
4 inicio = time.time()
5
6 # Ejecutar código a medir
7 resultado = bubble_sort(lista.copy())
8
9 # Detener cronómetro
10 fin = time.time()
11
12 # Calcular tiempo transcurrido
13 tiempo_total = fin - inicio
14 print(f"Tiempo: {tiempo_total:.4f} segundos")
```

Conceptos clave:

- `time.time()`: retorna tiempo actual en segundos
- Restar inicio de fin da el tiempo transcurrido
- Usar `.copy()` para no modificar datos originales

Medición de Rendimiento $\ni$ Solución 2 de Referencia:

Comparación de Algoritmos



```
1 import time
2 import numpy as np
3
4 def comparar_algoritmos(n):
5     # Generar datos aleatorios
6     datos = np.random.randint(
7         0, 10000, n
8     ).tolist()
9
10    # Medir Bubble Sort
11    inicio = time.time()
12    bubble_sort(datos.copy())
13    tiempo_bubble = time.time() - inicio
```

```
12    # Medir Merge Sort
13    inicio = time.time()
14    merge_sort(datos.copy())
15    tiempo_merge = time.time() - inicio
16
17    return tiempo_bubble, tiempo_merge
18
19 # Crear tabla comparativa
20 tamaños = [100, 500, 1000, 2000]
21 for n in tamaños:
22     t_b, t_m = comparar_algoritmos(n)
23     print(f"n={n:4d}: Bubble={t_b:.4f}s, "
24           f"Merge={t_m:.6f}s")
```

Discusión: Observa cómo la diferencia crece exponencialmente con n .

Aplicación: Catálogo Estelar

Aplicación: Catálogo Estelar $\ni$ Ejercicio 3:

Catálogo de Estrellas



Enunciado

- Generar 10,000 magnitudes estelares (distribución normal: media=8, desv=1.5)
- Ordenar usando Merge Sort
- Encontrar las 20 estrellas más brillantes (menor magnitud)
- Encontrar las 20 estrellas más débiles (mayor magnitud)
- Calcular la mediana del catálogo

Conceptos: Ordenamiento de datos reales, análisis estadístico.

Aplicación astronómica: Clasificación de estrellas por brillo aparente.

Aplicación: Catálogo Estelar $\ni$ Solución 3 de Referencia:

Catálogo Estelar



```
1 import numpy as np
2
3 # Generar catálogo simulado
4 np.random.seed(42)
5 magnitudes = np.random.normal(8, 1.5, size=10_000).round(2).tolist()
6
7 # Ordenar con Merge Sort
8 mags_ordenadas = merge_sort(magnitudes)
9
10 # Análisis
11 mas_brillantes = mags_ordenadas[:20]
12 mas_debiles = mags_ordenadas[-20:]
13 n = len(mags_ordenadas)
14 mediana = (mags_ordenadas[n//2] + mags_ordenadas[(n-1)//2]) / 2
15
16 print(f"Estrellas más brillantes (menor mag): {mas_brillantes[:5]}")
17 print(f"Estrellas más débiles (mayor mag): {mas_debiles[-5:]}")
18 print(f"Mediana del catálogo: {mediana:.2f}")
```

Discusión: En catálogos reales (millones de estrellas), algoritmos eficientes son cruciales.

Consolidación y Retroalimentación

Consolidación y Retroalimentación $\ni$ Puesta en Común - ¿Cómo les fue?

- ¿Fue difícil implementar Merge Sort recursivamente?
- ¿La diferencia de rendimiento fue la esperada?
- ¿Encontraron errores comunes? ¿Cómo los solucionaron?
- ¿Qué aplicaciones adicionales ven para estos algoritmos?

Objetivo

Aprender de las experiencias colectivas y consolidar comprensión.

Consolidación y Retroalimentación $\ni$ Errores Comunes con Merge Sort

Errores frecuentes

- Caso base incorrecto: olvidar `if len(lista) <= 1`
- División incorrecta: usar `/` en lugar de `//`
- Índices en merge: confundir `i` y `j`
- Olvidar elementos restantes: no usar `extend()`

En medición de tiempos

- No usar `.copy()`: modifica datos originales
- Medir solo una vez: resultados inconsistentes
- No calentar caché: primera ejecución más lenta

Consejo

Prueba con listas pequeñas primero, imprime resultados intermedios.

Conclusiones

- **Implementamos** Merge Sort usando recursividad
 - Complejidad $\mathcal{O}(n \log n)$ vs $\mathcal{O}(n^2)$
 - Dramáticamente más eficiente para listas grandes
- **Medimos** rendimiento empíricamente
 - Módulo `time` para cronometrar
 - Comparación cuantitativa de algoritmos
- **Aplicamos** a problemas astronómicos
 - Ordenamiento de catálogos estelares
 - Integración con búsqueda binaria

Habilidad adquirida

Capacidad de elegir e implementar algoritmos eficientes según escala de datos.

¡Nos vemos en la próxima sesión!